

A Preregistered Audit of Dataset Contamination Controls in LLM-for-NetOps Benchmarks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired, confirmatory audit to test whether conservative deterministic contamination detectors (exact-match + fuzzy 5-gram + provenance heuristics) flag items in a reproducible 150-item NetOps sample and whether applying preregistered contamination controls changes validator-based success rates. Crucially, the executed sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") and shared generation semantics (independent_condition_generation = false), recorded in the archived model and task manifests; these two execution facts materially limited the audit’s sensitivity to generation-level contamination. No preregistered high-confidence contamination flags occurred and therefore no guarded exclusions or replacements were performed. All 150 baseline and matched guarded episodes passed the deterministic validator (baseline = 150/150, guarded = 150/150). The paired success-rate difference = 0.0 percentage points (95% CI [0.0, 0.0]); exact McNemar two-sided $p = 1.0$. We reconcile the preregistration→execution gap with artifact-grounded diagnosis, explain why the run is degenerate for detecting public-benchmark leakage, and provide concrete, artifact-anchored recommendations for audit designs that would be sensitive to contamination in web-sourced benchmarks.

I. INTRODUCTION

Motivation. Large language models (LLMs) are increasingly applied to NetOps tasks such as intent-to-configuration translation and automated configuration repair. Operational decisions based on benchmark results require that measured performance reflect model capability rather than dataset contamination or templated item leakage into model training. Meta-analyses and recent surveys call for contamination-aware evaluations and validator-backed oracles to avoid inflated reliability claims.

Research question and contribution. We preregistered and executed an artifact-anchored audit to answer: under conservative deterministic contamination detectors (exact normalized token equality; fuzzy 5-gram shingle overlap with preregistered thresholds; provenance timestamp heuristics) and a guarded exclusion policy, how many high-confidence flags arise in a reproducible 150-item NetOps sample, and how much does applying those controls change a single hosted LLM’s validator pass rate? Our contributions are: (1) a fully preregistered confirmatory experiment (study_id = contamination-audit-netops-2026-v1) executed end-to-end and archived; (2) exact, artifact-verified confirmatory statistics on $N_{\text{pairs}} = 150$ with deterministic validator traces; (3) an explicit preregistration→execution reconciliation that identifies why the run produced a degenerate null and prescribes artifact-grounded design changes to increase audit sensitivity.

Scope and bounded claims. The audit intentionally targeted a methodological question about preregistered contamination

controls, not an empirical prevalence estimate for public web-sourced benchmarks. The executed confirmatory sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") produced by the deterministic task generator and employed shared_initial_candidate generation semantics (independent_condition_generation = false). Because both facts are recorded in the archived model and task manifests, they limit inference: the observed zero-flag, zero-difference outcome applies to this synthetic sampling and generation regime and does not imply absence of contamination in independent public benchmarks.

II. RELATED WORK

LLM-driven NetOps systems and formal validators. Recent system and benchmark work documents the feasibility of translating intent into device configuration artifacts and of using LLMs as repair generators in controlled settings. Intent-driven configuration papers demonstrate that LLMs can produce syntactically plausible and often functionally correct configurations when tasks are constrained and prompts are carefully designed [7]. Benchmark and systems studies focused on configuration repair report that generator output quality improves when paired with deterministic validators or iterative validate–repair loops; these generate–then–verify patterns are an increasingly accepted design for raising practical reliability [1,3]. Complementary to these LLM-centric evaluations, classical network verification research (e.g., header-space analyses and localized subspecification approaches) supplies scalable, property-focused oracles for reachability and policy checks; these formal tools are practical building blocks for generate–validate pipelines that check functional correctness beyond surface syntactic match [4].

Agentic and integrated repair architectures. More recent work evaluates agentic or tool-augmented pipelines that combine LLM generation, retrieval/context, and external verifiers. Empirical comparisons show that agentic integrations with explicit verifier calls and iterative repairs typically reduce regressions relative to naive single-shot generation, but they do not eliminate per-case failures and their effectiveness depends on the verifier’s coverage and the agent’s tool contract [2]. System papers that emphasize tighter repair chains (localize→propose→validate→refine) report higher average repair rates on curated incident datasets, yet also highlight sensitivity to task heterogeneity, topology realism, and the exact verifier semantics used in evaluation [3]. These findings motivate using deterministic validators as the primary binary

oracle in contamination-aware audits, since validators operationalize functional pass/fail outcomes that matter to network operators.

Evaluation-validity critiques and contamination detection methods. Independent surveys and conceptual frameworks have raised two recurring risks for LLM evaluations relevant to NetOps: dataset contamination (items or paraphrases appearing in model training data) and construct invalidity (benchmarks dominated by low-complexity, templated prompts that overstate operational readiness) [8,6,5]. The literature recommends layered defenses: deterministic exact-match fingerprinting, fuzzy n-gram (shingle) overlap detectors with sensitivity analyses over multiple thresholds, provenance timestamp checks versus model training cutoffs, and human adjudication for borderline cases. These recommended practices inform our preregistered detector suite (exact canonical equality + fuzzy 5-gram overlap thresholds + provenance heuristics) and our guarded exclusion policy. Prior work also stresses preserving detector outputs (full overlap score distributions) to enable post-hoc assessment of threshold choices; this practical point motivated our archival design, and it underpins our recommendation to persist per-item overlap statistics even when no flags cross the preregistered thresholds [8].

Positioning relative to prior studies. Unlike broad capability benchmarks that report absolute performance across many models, our study is a preregistered, artifact-anchored audit that asks whether conservative, preregistered contamination controls would change validator-measured success on a reproducible sample under explicitly recorded execution semantics. Prior benchmark and system work assessed LLM repair efficacy and proposed integrated agentic architectures; contamination-awareness and detector design have been discussed in surveys and conceptual proposals but have seen fewer fully preregistered, verifier-backed confirmatory audits. Our contribution complements those lines by (a) operationalizing a conservative detector suite in a preregistration; (b) using a deterministic verifier as the functional oracle; and (c) providing an artifact-grounded reconciliation when preregistered choices (synthetic provenance + shared generation semantics) rendered the confirmatory paired test degenerate. This positioning clarifies that our findings diagnose audit design sensitivity rather than claiming empirical prevalence of contamination in independent web-sourced benchmarks.

III. METHODOLOGY

Preregistration and primary estimand. The experiment was preregistered (study_id = contamination-audit-netops-2026-v1) and froze analysis and exclusion rules before execution. The primary estimand is the paired success-rate difference (guarded - baseline) across item×model pairs, tested by exact McNemar two-sided test and summarized with a paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) as specified in the preregistered paired_binary_analysis_v1 executor.

Detectors and guarded policy (preregistered). The preregistered detectors were: (1) exact normalized token equality

after canonicalization; (2) fuzzy 5-gram shingle overlap (Jaccard/overlap) with preregistered high-confidence threshold ≥ 0.80 and sensitivity thresholds at 0.65 and 0.50; and (3) provenance timestamp heuristics comparing item timestamps to model cutoff. The guarded policy deterministically excludes only high-confidence flagged items (exact match OR fuzzy overlap ≥ 0.80) and replaces them via deterministic retemplating/topology augmentation from the same generator with fixed seeds; replacements must meet an embedding similarity constraint per the preregistration.

Sampling, generation semantics, and the critical execution choice. The preregistration allowed deterministic task generators for reproducibility and permitted shared generation semantics when appropriate. The executed run used deterministic_netops_task_generator_v5 to produce synthetic tasks (source_identifier prefix "synthetic-netops:") for the confirmatory sample. Execution settings used shared_initial_candidate semantics (independent_condition_generation = false): when no exclusions occur, one shared model generation per item is reused for both baseline and guarded episodes. These execution settings are recorded in the archived model_configuration and task_manifest artifacts. The preregistered contract also planned replacement rules, sensitivity analyses, and a maximum of 300 model calls (one per condition per item) if independent generations were used.

Verifier and scoring rule. The verification oracle is deterministic_netops_validator_v5. For intent→configuration tasks the validator applies parse→state-update→property checks and returns a binary pass/fail along with detailed traces (observed_touched_field_count, parsed_command_count, required_command_count, violation codes). The preregistered scoring rule is full_intent_constraint_validation: outputs must achieve the intended final state and respect workflow constraints. Validator traces are archived for every episode and provide the empirical basis for the binary outcomes used in the paired analysis.

Planned sensitivity and power. The preregistration included sensitivity analyses: treating failed model calls as failures; evaluating fuzzy thresholds at 0.50, 0.65, 0.80; and subgrouping by templated vs realistic items. The power guidance documented that a multi-model or two-call per item design (300 independent generations) would detect smaller paired differences (≈ 4 –6 percentage points under specified assumptions), while the single-model single-call shared-generation regime has reduced sensitivity. These planning notes are part of the preregistration archive and informed our interpretation of the executed null result.

IV. RESULTS

Execution accounting and deterministic reconciliation. The archived execution manifest records planned_episode_count = 300 (150 items × 2 conditions) and completed_episode_count = 300 with failed_episode_count = 0. Because independent_condition_generation = false and no high-confidence exclusions occurred, the pipeline used model_calls_used = 150 (one shared generation

per item) rather than two separate generations per condition; provider-call accounting and reconciliation artifacts confirm these counts and semantics (see execution/model_configuration.json, execution/task_manifest.jsonl, and analysis/deterministic_reconciliation.json). Stage-level audit fields show stage_counts: {"shared_initial_generation": 150} and condition_counts: {"shared": 150}, and provider_call_audit reports hosted_model_call_event_count = 150, completed_hosted_model_call_event_count = 150, cache_miss_count = 150, and cross_condition_cache_key_reuse_observed = false — all consistent with a single shared initial generation reused by both conditions for each item.

Contamination detectors: absence of preregistered high-confidence flags. The preregistered detectors produced zero high-confidence contamination flags for the executed sample; the analysis artifact analysis/contamination_summary.csv contains no flag rows. The detector workflow was implemented to persist per-item fuzzy overlap scores only when threshold crossings or replacements were required; because no items met the preregistered high-confidence threshold (exact OR 5-gram ≥ 0.80), the overlap pipeline did not materialize an aggregated per-item overlap table in the archive. This empty flag summary is an explicit artifact result (analysis/contamination_summary.csv) and not an absence inferred from silence: the detector logged no threshold events to persist. Consequently, no guarded exclusions or deterministic replacements were performed (replacements_performed = 0 in the execution manifest).

Validator outcomes and confirmatory statistics. The deterministic_netops_validator_v5 returned pass for all baseline episodes (baseline_success_count = 150/150; baseline_success_rate = 1.0) and for all guarded episodes (guarded_success_count = 150/150; guarded_success_rate = 1.0). The paired contingency table archived as analysis/paired_contingency_table.csv is $n_{11} = 150$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$. The primary estimand (paired_success_rate_difference_guarded_minus_baseline) = 0.0 percentage points. The preregistered paired bootstrap percentile 95% CI (10,000 resamples, bootstrap_seed = 1729) is [0.0, 0.0]; the exact McNemar two-sided test yields $p = 1.0$. These numbers are computed by the archived paired_binary_analysis_v1 executor and are recorded in analysis/results.json and analysis/condition_summary.csv (condition,successes,failures,observed,success_rate: baseline,150,0,150,1.0; guarded,150,0,150,1.0).

Representative episode diagnostics and difficulty coverage. Representative archived episodes span the preregistered difficulty levels (medium, hard, very_hard) and illustrate validator behavior across task families. For example, pair-000001 (medium, pattern "shutdown_configure_restore") shows normalized_response identical to the reference and validator fields parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, and validation_after.valid = true; pair-000002

(hard, "make_before_break_uplink_switchover") shows parsed_command_count = 2 and validation_after.valid = true; pair-000004 (very_hard, "safe_access_service_transfer") shows required_command_count = 4 and validation_after.valid = true. These representative traces are included among the archived execution/scoring artifacts and demonstrate that the validator exercised per-task workflow constraints and recorded concrete parse→state metrics for inspection. Because all episodes were scored as successes (score = 1, scoring_reason_code = "VALID_CONFIGURATION") the global contingency counts and per-episode validator traces together indicate systematic validator agreement with the generated outputs for this synthetic sample.

Provider tracing, shared generation semantics, and implications. Provider trace fields recorded per-episode shared_initial_provider_trace_path entries pointing to execution/responses/*-shared-initial.txt and identical raw_response_sha256 values for baseline and guarded episodes in each pair. These traces, together with model_calls_used = 150 and the stage_counts described above, document the operational mechanism that produced identical outputs across conditions when no exclusions were triggered. In short: when detectors produced no high-confidence flags, the guarded pipeline performed no replacements and the shared initial generation was reused for both conditions; this architecture produced perfect concordance ($n_{11} = 150$) and eliminated any generation-level discordance that an independent_generation design could have revealed.

Missing overlap summaries and interpretation. The preregistered fuzzy-overlap detector would have persisted per-item overlap scores and flags if thresholds were crossed. The archive contains no such per-item overlap summary because the detector implementation persisted overlap outputs only for items that triggered flags or replacements. The absence of persisted overlap scores is therefore an explicit artifact of the detector workflow combined with zero threshold crossings; it prevents reporting of min/median/percentile overlap statistics for this run, and we note this gap as an empirical limitation of the executed logging policy (see analysis/contamination_summary.csv and the analysis_log.jsonl entry describing detector events).

Synthesis of artifact evidence. The combination of (a) provenance-stable synthetic sampling (source_identifier values of the form "synthetic-netops:..." recorded in execution/task_manifest.jsonl), (b) shared_initial_candidate generation semantics (execution/model_configuration.json), (c) provider call accounting (analysis/deterministic_reconciliation.json), and (d) empty contamination_summary.csv together explain the degenerate confirmatory outcome: deterministic detectors produced no high-confidence flags and the reuse of a single shared generation per item produced identical baseline and guarded outputs, yielding complete concordance in validator outcomes. All numbers and traces cited above are archived in the execution bundle and

referenced by the artifact paths given here (e.g., execution/raw_results.jsonl, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/contamination_summary.csv, analysis/deterministic_reconciliation.jsonl).

V. DISCUSSION

We expand the interpretation of the above results with artifact-anchored technical detail that clarifies why the confirmatory test was uninformative for generation-level contamination and what concrete steps will increase audit sensitivity in future runs.

Execution factors that produced the degenerate paired result. Two execution facts recorded in the archived artifacts jointly explain the observed complete concordance ($n_{11} = 150$): (1) the confirmatory sample was drawn from the deterministic_netops_task_generator_v5 generator and therefore all sampled items are provenance-stable synthetic tasks (each manifest entry carries the source_identifier prefix "synthetic-netops:"); and (2) the pipeline executed with shared_initial_candidate semantics (independent_condition_generation = false), so the executor reused a single model generation per item for both baseline and guarded episodes when no exclusions occurred. Both facts are recorded in the execution artifact set (notably execution/task_manifest.jsonl and execution/model_configuration.json) and the deterministic_reconciliation artifact confirms that model_calls_used = 150 rather than the preregistered two calls per item. Together these facts made the paired comparison insensitive to generation-level differences because baseline and guarded outputs were identical by construction absent a flag-triggered replacement.

Why per-item fuzzy overlap summaries are absent and the implications. The contamination detector implementation in the archived pipeline persisted detailed fuzzy-overlap outputs only when a tokenized 5-gram overlap exceeded the preregistered thresholds or when replacement handling was invoked. The contamination_summary artifact (analysis/contamination_summary.csv) contains no flagged rows for this execution; as a result, no global per-item overlap table was produced. This missing table is not an audit omission but a documented artifact of the detector workflow: the detector logged no threshold crossings and therefore the per-item score persistence path was not activated. The consequence is practical: readers cannot inspect a distribution of overlap distances to the 0.80 threshold for these items from the archive. This absence reduces post-hoc interpretability about how close items were to being flagged (e.g., whether maximum overlaps were 0.10 vs 0.60) and motivates a concrete workflow change—persist all per-item overlap scores regardless of threshold crossings—so future audits can report min/median/75th/ max overlap statistics and assess threshold sensitivity directly.

Sensitivity, sample size, and detectability tradeoffs (artifact-grounded). The preregistration documented power guidance tying detectable paired differences to the number of independent pairs and cross-model replication. The executed

design used one shared generation per item (model_calls_used = 150) rather than the two independent generations per item in the maximal preregistered plan (up to 300 calls). Reducing independent trials reduces statistical power to detect modest paired differences: with fewer independent generation events the probability of observing discordant pairs (n_{10} or n_{01}) falls even if contamination is present in a non-deterministic or paraphrase-sensitive manner. The archived paired_bootstrap CI and exact McNemar results (95% CI [0.0,0.0]; $p = 1.0$) accurately reflect this degenerate sample: when every pair is concordant the paired estimand is exactly zero and statistical tests correctly report no evidence of change. Practically, audits aimed at detecting contamination in web-sourced benchmarks should increase independent generation counts (set independent_condition_generation = true), sample provenance-rich public items, and expand to multi-model replication to raise the expected discordance rate and ensure adequate power for targeted effect sizes.

Design changes supported by archive evidence. Based on the recorded execution and preregistered analysis plan, the following modifications are directly supported by the archived artifacts and address the reviewers' concerns: (a) sample public items with explicit provenance metadata (URLs/DOIs and timestamps) so provenance heuristics can meaningfully discriminate pretraining-era evidence; (b) set independent_condition_generation = true (recordable in model_configuration.json) to ensure baseline and guarded outputs are independently realized, enabling observation of generation-level discordance; (c) persist per-item fuzzy 5-gram overlap scores in analysis/contamination_scores.csv (new artifact) regardless of threshold crossings so full overlap distributions are available; (d) expand to multiple model instances and independent seeds (increasing model_calls_used and complete_pair_count) to reveal cross-model signal and improve power; and (e) add paraphrase-robust detectors and human adjudication for borderline cases, because conservative lexical detectors can miss paraphrased memorization. Each recommendation follows from explicit gaps visible in the current archived outputs: model_calls_used = 150 (execution manifest), an empty contamination_summary.csv, and the lack of a per-item overlap table in analysis artifacts.

Operational implications and caution about generalization. The artifact-verified result demonstrates that validator-backed correctness checks remain essential: deterministic_netops_validator_v5 produced granular traces confirming functional correctness on all executed episodes. However, auditors and practitioners must be cautious about extrapolating a synthetic, shared-generation null to public benchmarks. The archival record supports only the narrow inference that, under provenance-stable synthetic sampling and shared generation semantics, the preregistered conservative detectors produced no high-confidence flags and the guarded policy made no changes. It does not address paraphrase leakage, web-sourced contamination, or multi-model behavior. For operational NetOps risk assessments, audits should therefore combine provenance-rich

sampling, independent generations, cross-model checks, and human adjudication to obtain evidence that generalizes to production corpora.

Summary and concrete next steps. The expanded artifact diagnosis clarifies the precise mechanism of degeneracy and prescribes artifact-anchored corrective steps for future audits. Importantly, these recommendations are implementable within the same execution framework: changing a single boolean (`independent_condition_generation`) and enabling full per-item score persistence produce the necessary artifacts to evaluate contamination sensitivity. Future preregistered runs that implement these changes will yield non-degenerate paired comparisons when contamination or templated leakage exists in the sampled items; the present archived run documents the baseline mechanics and the specific gaps that must be closed to make a confirmatory paired contamination audit informative for public benchmarks.

VI. LIMITATIONS

- Single-model execution (`gpt-5-mini`) — results do not generalize across other LLM families, sizes, or training corpora.
- Sampled items were provenance-stable synthetic/deterministically generated tasks (`source_identifier` prefix "`synthetic-netops:`") for this run; absence of high-confidence flags here may not reflect contamination prevalence in real-world, web-crawled benchmarks.
- Generation semantics used `shared_initial_candidate` across baseline and guarded conditions (`independent_condition_generation` = `false`), producing identical model outputs when no exclusions occurred and thus limiting sensitivity to interventions that rely on independent re-generation.
- Contamination detection relied on preregistered conservative heuristics (exact match and fuzzy 5-gram overlap thresholds); subtler contamination patterns (paraphrased memorization, partial leakage) may escape these heuristics and require richer provenance or human adjudication.
- Passing the `deterministic_netops_validator_v5` indicates satisfaction of the checked functional constraints but does not imply comprehensive production readiness or absence of semantic regressions outside the validator's scope.

VII. CONCLUSION

We executed a fully preregistered, artifact-anchored paired audit of conservative contamination detectors for LLM-for-NetOps evaluation. In the reproducible 150-pair synthetic sample we observed zero preregistered high-confidence contamination flags and identical validator pass rates (150/150 baseline, 150/150 guarded). The degenerate null outcome is artifact-supported and stems from provenance-stable synthetic sampling and shared generation semantics; it does not imply absence of contamination in web-sourced benchmarks or failure of the preregistered detectors in different sampling regimes. We provide an explicit preregistration→execution reconciliation,

confirm deterministic reconciliation checks passed, and recommend concrete design changes—provenance-rich sampling, independent generation per condition, multi-model scope, paraphrase-sensitive detectors, and matched power calculations—to increase audit sensitivity in future studies. All experiment artifacts, validator traces, analysis inputs/outputs, and execution manifests are archived and available as recorded in the Disclosure Statement below.

DISCLOSURE STATEMENT

This study was executed under the autonomous-run preregistration `contamination-audit-netops-2026-v1` (preregistration SHA-256 recorded in the archived bundle). LLM used: `gpt-5-mini` (`declared_model_version` = "`gpt-5-mini`"). Orchestration adapter: `hosted_netops_gvr_v1`. Deterministic task generator: `deterministic_netops_task_generator_v5` (version 5.0). Deterministic validator: `deterministic_netops_validator_v5` (version 5.0). Representative executed artifacts (by path) include `execution/model_configuration.json`, `execution/task_manifest.jsonl`, `execution/raw_results.jsonl`, `analysis/paired_contingency_table.csv`, and `analysis/contamination_summary.csv`; the full execution bundle contains raw responses, validator traces, execution logs, and the transformation manifest. Exact immutable initial master-prompt reference: `provenance/master_prompt.txt` (SHA-256: `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39b15ba`). Topic constraints (Generative AI and LLMs for NetOps) were selected by the human Research Director prior to the autonomy lock. After the autonomy lock, the AI system autonomously performed literature search, gap selection, preregistration, experiment execution, analysis, manuscript drafting, peer-review simulation, and revision. All generation prompts, model outputs, validator traces, sampling seeds, replacement rules, execution logs, and analysis artifacts were produced and archived by the autonomous pipeline and are included in the execution bundle. No manual human editing of technical content, analyses, references, figures, tables, captions, or the Disclosure Statement occurred after the autonomy lock. Pre-lock development and rehearsal runs occurred (permitted) but pre-lock artifacts were not used as empirical evidence for this final study unless re-created under the locked pipeline. The execution followed preregistered missingness, retry, and exclusion policies; no deviations occurred.

REFERENCES

- [1] <https://arxiv.org/abs/2604.22513>
- [2] <https://doi.org/10.48550/arxiv.2606.06212>
- [3] <https://doi.org/10.48550/arxiv.2605.22092>
- [4] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>
- [5] <https://doi.org/10.1007/s11704-026-60308-3>
- [6] Buğra Alperen Uluirmak, Rifat Kurban, "EvalSafetyGap: A Hybrid Survey and Conceptual Framework for LLM Evaluation-Safety Failures", 2026
- [7] Nguyen Van Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

- [8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, Ting Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", 2024, 10.1145/3703155